# Rattrapage — Épreuve : le module Ticket

**KFOKAM48 — Batch 2 — Phase 3 · Séances Full Stack · Rattrapage des séances 1, 2 et 3**
**Durée : 2 heures · Backend Spring Boot et PostgreSQL · Frontend Angular ou React, au choix**

---

## Le contexte

Cette épreuve reprend en une seule fois ce que les séances 1, 2 et 3 évaluaient : faire tourner un environnement, construire une API avec ses règles de gestion, lui donner une interface.

Un service de support veut suivre les demandes de ses utilisateurs. Un **ticket** a un titre, une description, un statut et une date. On peut en créer, les lister et les fermer. Vous partez d'un **dossier vide** : ce projet n'a aucun rapport avec la Gestion de Bibliothèque.

**Aucun code ne vous sera donné.** La documentation officielle et Swagger sont vos seules sources. Les parties se suivent : un écran branché sur une API qui ne fonctionne pas ne vaut rien.

---

## Ce qu'il faut livrer

Quatre éléments, dans cet ordre :

1. Un **environnement** qui tourne (20 min)
2. Une **API Ticket** de trois endpoints (45 min)
3. Un **écran** qui la consomme (40 min)
4. Une **branche** et une Pull Request, puis le passage devant le formateur (15 min)

---

## A. L'environnement (20 min)

Dans un dossier neuf. Aucune commande ne vous est donnée : la documentation officielle suffit.

1. Lancez **PostgreSQL dans un conteneur Docker**. La base ne s'installe pas sur votre machine.
2. Créez le **backend Spring Boot**, connectez-le à cette base et démarrez-le. Lisez les logs : ils doivent être sans erreur.
3. Créez le **frontend** (Angular ou React) et démarrez-le.
4. Une fois la partie B faite, **retrouvez dans la table PostgreSQL** un ticket créé depuis Swagger, avec une requête SQL.

---

## B. Le backend (45 min)

### L'entité à créer

### `Ticket`

| Champ | Type | Contraintes |
|---|---|---|
| id | identifiant | généré |
| titre | texte | obligatoire, 100 caractères maximum |
| description | texte | facultative |
| statut | énumération | OUVERT, EN_COURS, FERME |
| dateCreation | date et heure | générée par le serveur, jamais fournie par le client |

### Les règles de gestion

| Réf. | Règle |
|---|---|
| **RG-01** | Le **titre** est obligatoire et ne dépasse pas 100 caractères. Un ticket est créé avec le statut OUVERT. |
| **RG-02** | Deux tickets ne peuvent pas avoir le **même titre**. |
| **RG-03** | Un ticket ne peut être fermé que s'il est **OUVERT** ou **EN_COURS**. Un ticket FERME ne change plus d'état. |

### Les endpoints attendus

| Verbe | Chemin | Rôle | Succès | Erreurs |
|---|---|---|---|---|
| POST | /api/tickets | Créer un ticket | 201 | 400, 409 |
| GET | /api/tickets | Lister, filtrable par statut | 200 | — |
| PATCH | /api/tickets/{id}/fermer | Fermer un ticket | 200 | 404, 409 |

Le client n'envoie que `titre` et `description`. Tout le reste est déterminé par le serveur.

### Ce qui est exigé

**Architecture.** Entité, repository, service, contrôleur. Aucune logique métier dans le contrôleur.

**DTO.** L'entité Ticket ne sort jamais du service. Un DTO en entrée, un DTO en sortie.

**Validation.** Un titre absent ou de plus de 100 caractères est refusé en 400, avec un message qui dit pourquoi.

**Gestion des erreurs.** Un identifiant inconnu renvoie 404. Une règle violée (RG-02, RG-03) renvoie 409 avec un message qui nomme la règle — pas « erreur serveur ».

**Documentation.** Les trois endpoints apparaissent dans Swagger, avec leurs codes de retour.

---

## C. Le frontend (40 min)

Vous consommez votre propre backend. S'il est incomplet, terminez-le d'abord. Un **écran unique**, comportant une liste, un formulaire de création et une action de fermeture.

### La liste

**Colonnes affichées :** titre, statut, date de création, action.

**Un filtre par statut :** tous, `OUVERT`, `EN_COURS`, `FERME`.

## Les quatre états à gérer

Un écran qui n'affiche que le cas nominal est incomplet.

| État | Ce que l'utilisateur doit voir |
|---|---|
| **Chargement** | Un indicateur visible pendant l'appel — jamais un écran figé |
| **Données** | Le tableau rempli |
| **Liste vide** | Un message explicite : « Aucun ticket », pas un tableau vide sans en-tête |
| **Erreur** | Un message compréhensible et un moyen de réessayer |

**Pour tester l'état d'erreur :** arrêtez votre backend et rechargez la page. L'écran doit expliquer que le serveur est injoignable, pas rester bloqué sur le chargement.

## Le formulaire de création

Deux champs : **titre** et **description** (facultative). Le bouton reste inactif tant que le titre est vide. Ne limitez pas la longueur du titre côté client : le message de refus doit venir du serveur.

**Après un succès :** la liste se rafraîchit et le nouveau ticket apparaît, sans rechargement de la page.

## Le traitement des refus

| Situation | Code reçu | Ce qui doit s'afficher |
|---|---|---|
| Titre de plus de 100 caractères | 400 | Le message de validation du serveur, à côté du formulaire |
| Titre déjà utilisé | 409 | Le message du serveur, lisible, à côté du formulaire |
| Fermeture d'un ticket déjà FERME | 409 | Le message du serveur |

**Interdit :** un `alert()`, un message générique du type « une erreur est survenue », ou pire, un échec silencieux où rien ne se passe quand l'utilisateur clique.

## La fermeture

Un bouton par ligne, qui appelle `PATCH /api/tickets/{id}/fermer`.

- Il n'apparaît que si le statut est `OUVERT` ou `EN_COURS`
- Une confirmation est demandée avant l'appel
- En cas de succès, le statut se met à jour dans la liste
- En cas de `409`, le message du serveur s'affiche

## Les exigences d'architecture

**Un service dédié aux appels API.** Aucun `fetch`, `axios` ou `HttpClient` appelé directement depuis un composant.

**Découpage en composants.** Au minimum : un conteneur qui gère l'état et les appels, un composant de liste, un composant de formulaire.

**Aucune donnée codée en dur.** Tout vient de l'API.

**Interface en français.**

---

## D. Git, Pull Request et passage devant le formateur (15 min)

Une branche `rattrapage/prenom-nom` avec **au moins trois commits** aux messages explicites, poussée sur le dépôt, et une **Pull Request** vers `main`.

Dans la description : **deux captures d'écran** — la page Swagger des trois endpoints, et un refus `409` affiché dans l'écran.

Le formateur passe à votre poste. **Vous manipulez vous-même** votre écran, et il provoque des erreurs en direct, pas seulement le cas qui fonctionne. Préparez votre environnement à l'avance : backend démarré, base contenant au moins un ticket déjà créé.

---

## Barème

| Élément | Points |
|---|---|
| Environnement : PostgreSQL Docker, backend et frontend démarrés, ticket retrouvé en base | 3 |
| Backend : couches et DTO en entrée et en sortie | 1 |
| Backend : création, liste et filtre par statut | 2 |
| Backend : fermeture (200 et 404) | 1 |
| Backend : refus 400 et 409 explicites | 3 |
| Frontend : liste, colonnes et filtre par statut | 1 |
| Frontend : les quatre états | 2 |
| Frontend : formulaire fonctionnel | 1 |
| Affichage lisible des erreurs du serveur (400, 409) | 2 |
| Frontend : fermeture avec confirmation et mise à jour de la liste | 1 |
| Architecture : service isolé, composants découpés | 1 |
| Git : branche, trois commits, Pull Request décrite | 2 |
| **Total** | **20** |

Validation de l'épreuve à partir de **10 points**. Le traitement des erreurs reste le poste le mieux doté : afficher des données est facile, gérer ce qui rate est ce qui sépare un écran d'école d'un écran livrable.

---

## Si vous finissez en avance

- Pagination de la liste
- Badge de couleur par statut
- Tri par date de création
- Compteur de tickets ouverts